

Assignment 1 report

김환욱

- **Task 0**
 - **0.1**

What I implemented

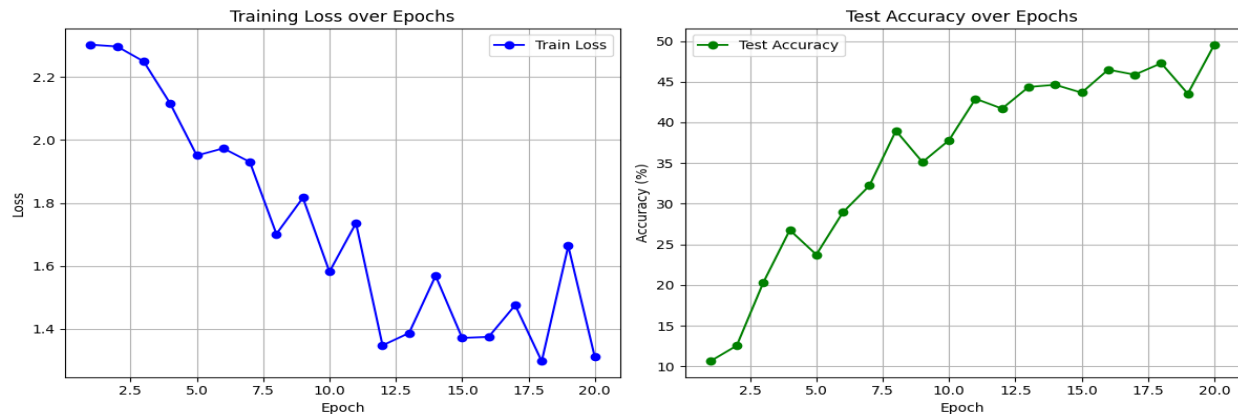
- One optimizer step per entire training set ($B = 50,000$).
- Forward once over the whole set, compute loss, do a single backward(), then step().

Why OOM is expected (analysis)

Activation memory, not parameters, dominates. Parameters+grads $\sim 2.4M \times 4$ bytes $\times 2 \approx \sim 19MB$ (weights+grads).

But activations scale with batch size. For the first big conv feature map:

- Shape after conv2: $[B, 64, 32, 32]$
- Elements = $50,000 \times 64 \times 32 \times 32 = 1.6384e9$
- FP32 = 4 bytes $\rightarrow \sim 6.55$ GB for just one activation tensor.
- You also keep multiple intermediate activations for backprop and additional layers (plus cuDNN workspaces).
- Backward typically needs the saved forward activations; many frameworks also need temporary buffers.
- Realistic peak easily exceeds tens of GB, hence OOM on typical 8–16GB GPUs.



Conclusion: FBGD is memory-prohibitive mainly due to activation storage, not parameter size.

○ 0.2

What I implemented

- Train the same CNN for 20 epochs with:
- SGD (lr=0.01),
- SGD+Momentum (lr=0.01, momentum=0.9),
- Adam (lr=0.001).
- Dataloaders with batch size 256, standard train/eval, record train loss and test accuracy each epoch, plot both curves.

Typical behavior you should observe

- SGD: Slowest early improvement; can catch up later if tuned.
- SGD+Momentum: Faster convergence than plain SGD; often better final accuracy.
- Adam: Fastest initial loss drop; final accuracy may be comparable or slightly below momentum-SGD for vision CNNs (dataset-dependent).

- Task 1

○ 1.1

What I implemented (single-GPU simulation)

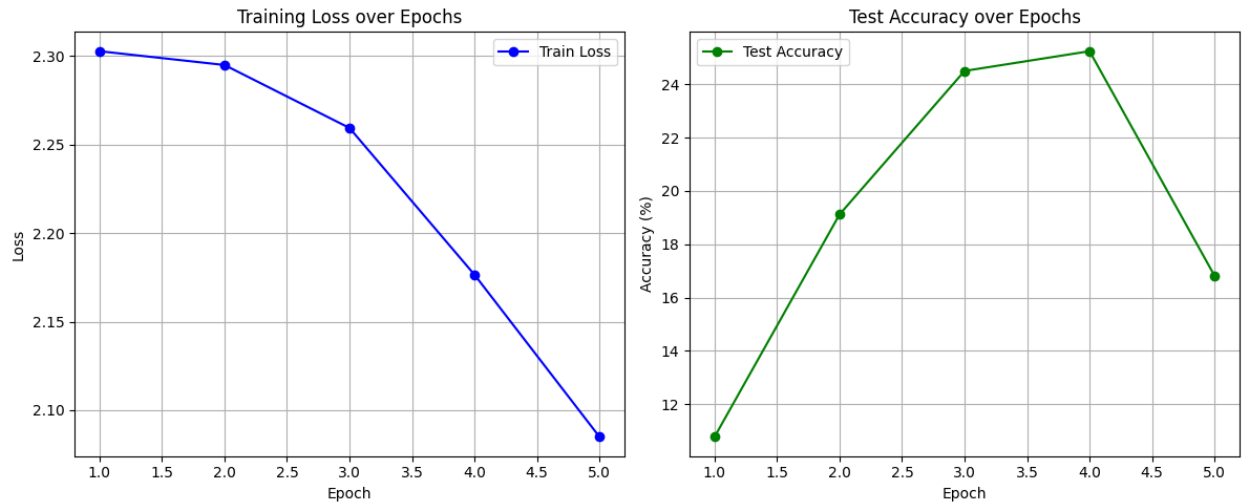
- Split the dataset into 4 equal shards (~12,500 imgs each).
- For each worker:
 1. Move its entire shard to device sequentially (or in micro-chunks if needed).
 2. Forward + compute loss + backward() to get local grads.
 3. Collect all workers' grad lists and average element-wise.
 4. Load averaged grad into the server model → one optimizer.step() per global full-batch.
- Repeat for 5 epochs.

Why this makes FBGD feasible

- Peak activation memory depends on the largest batch processed at once.
- With 4 shards, each worker's forward/backward sees only $B/4 = 12,500$ images at a time → peak activation cut $\sim 4\times$ vs full 50k.
- Workers are processed sequentially on the single GPU; we only store their grads (tiny) and throw away activations when each backward finishes.
- Averaging grads across workers is mathematically the same as a true full-batch gradient on the entire dataset

Why DP is essential for scaling

- Larger models/datasets → activation memory explodes with batch size.
- Horizontal scale (more workers/GPUs) = split data, reduce peak per device.
- All-reduce mean of grads keeps the update exactly equal to global full-batch (or to global mini-batch if using per-worker mini-batches).
- Enables throughput and memory scaling without changing the optimization objective.



○ 1.2

What I implemented

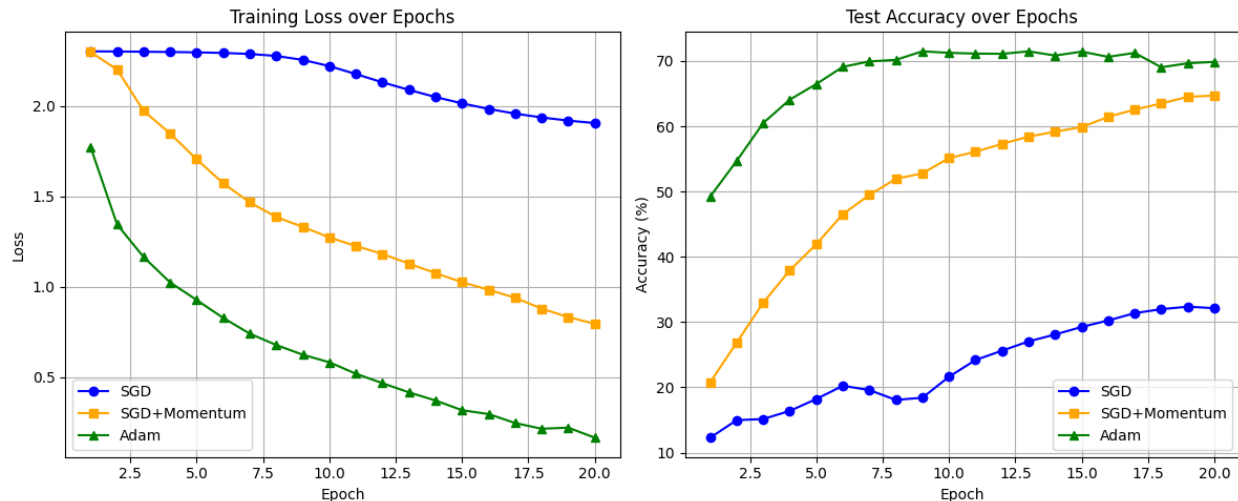
- Simulate 4 workers, each trains on mini-batch=64 from its shard (or from the same shuffled dataset partitioned per step).
- For each global step:
 1. Each worker does forward/backward on its mini-batch of 64, producing local grads.
 2. Average the grads across workers (simulated all-reduce mean).
 3. Update server model once (optimizer.step()).

This is identical to DDP behavior for data parallel mini-batch training with each g_i computed on 64 samples; total effective batch = 256.

What to compare (expected trends)

- Accuracy vs epoch curves for SGD / SGD+Mom / Adam under:
 - Centralized (Task 0-2, B=256),
 - DP with 4×64 (this task).
- With identical global batch and proper loss normalization, curves should be very close. Minor differences can arise from:
 - Data shuffling differences per worker,

- Floating-point non-associativity in grad averaging,
- Any small implementation choices (e.g., gradient accumulation ordering).



- Task 2

What I implemented (single-GPU simulation)

- Split the model into $K=4$ sequential partitions (stages).
 - For each mini-batch ($B=256$):
1. Forward goes $P0 \rightarrow P1 \rightarrow P2 \rightarrow P3$, strictly sequential.
 2. Compute loss at the end.
 3. Backward is triggered once; autograd sends gradients $P3 \rightarrow P2 \rightarrow P1 \rightarrow P0$ in reverse.
- At any time step, only one stage is active $\rightarrow$ this simulates naive pipeline's bubbles (idle time) if it were on multiple GPUs.

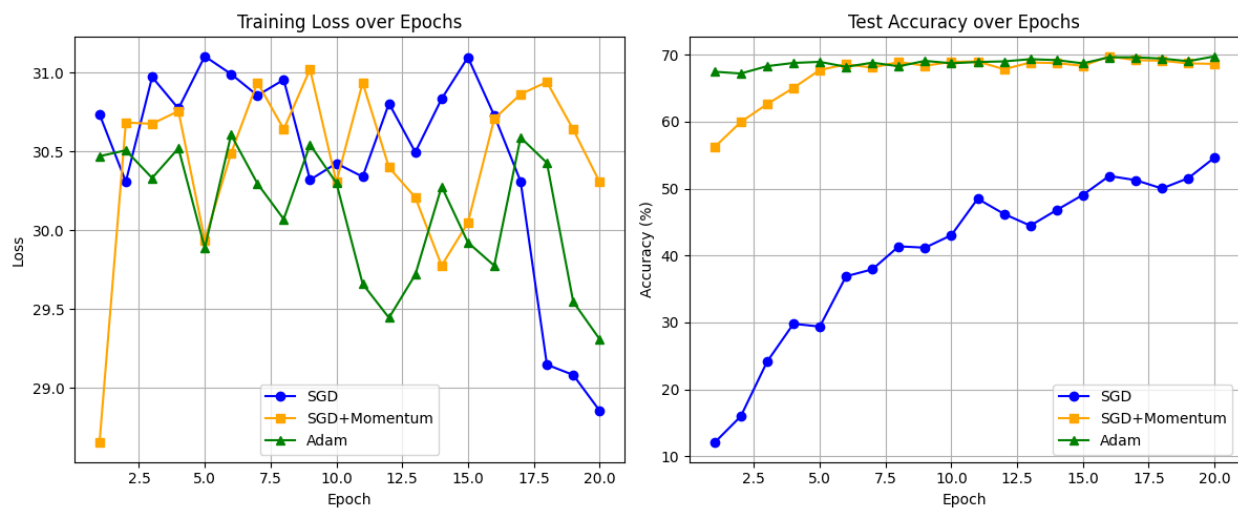
Utilization (theoretical intuition)

- In the naive case, if you imagine time segments per stage, only $1/K$ of stages are active at a time.

- Your utilization logging (sum of stage times $\div$ total step time) should show a low % (e.g., $\approx 25\%$ for $K=4$; real numbers vary because it's a single GPU executing sequentially).

Optimizer comparison

- Train 20 epochs with SGD / SGD+Mom / Adam as before.
- Plot Training Loss vs Epoch and Test Accuracy vs Epoch.
- Expect broadly similar optimizer ranking as centralized training, but throughput/utilization is worse conceptually due to bubbles.



- Task 3

What I implemented (single-GPU simulation)

- Split each mini-batch $B=256$ into $M=4$ micro-batches (64 each).
- Forward (fill) is executed in time steps $t=0 \dots (M+K-2)$:

at time t , stage s works on micro $mb=t-s$ if $0 \leq mb < M$.

- After all forwards finish, do all-backward (drain) in micro-batch reverse order.
- Record an approx utilization per step as $(\text{active_stages} / K) * 100$.

Why GPipe reduces “bubbles”

- Naive: at any step, only 1 stage is active $\rightarrow$ large idle time.
- GPipe: after warm-up, multiple stages process different micro-batches concurrently (conceptually).
- Theoretical steady-state utilization (idealized) $\sim \frac{M}{M+K-1}$.

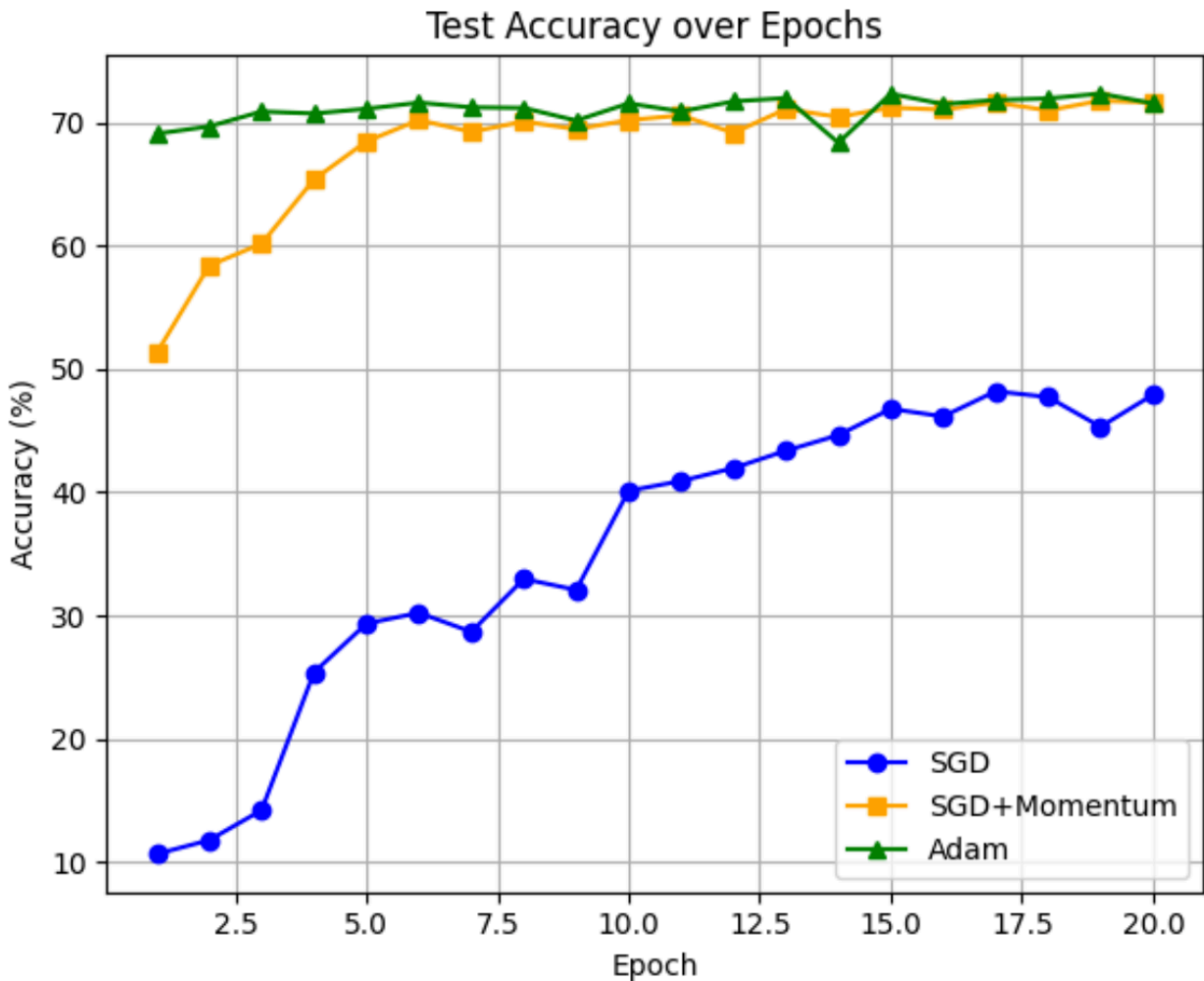
For $K=4$, $M=4 \rightarrow \frac{4}{7} \approx 57\%$ (vs naive $\approx 25\%$).

- You should observe higher simulated utilization than naïve and similar (often slightly better) learning dynamics, because the optimization math is identical (only execution schedule changes).

Optimizer comparison

- Repeat SGD / SGD+Mom / Adam for 20 epochs with GPipe schedule.
- Plot Loss/Accuracy curves.
- Compare Centralized vs Naive vs GPipe for each optimizer.

Expect similar accuracy across methods (they all compute the same gradients per global batch, up to micro-batch scaling and numerical ordering). GPipe improves throughput/util in real multi-GPU settings by reducing bubbles.



Key Takeaways

1. OOM in full-batch stems from activation blow-up, not parameter size.
2. Data Parallelism makes FBGD feasible by shrinking peak batch per device, then averaging grads.
3. Mini-batch DP (4×64) is equivalent to centralized $B=256$; curves should be very close.
4. Naive pipeline has severe bubbles (low utilization).

5. GPipe uses micro-batches to overlap stages and reduce bubbles; on real multi-GPU, that means higher throughput with the same math.